Self-Pruning Neural Network on CIFAR-10

Tredence AI Engineering Internship – Case Study

Author: Writick Parui

GitHub: https://github.com/writickp3-ctrl/Self-Pruning-Neural-Network-on-CIFAR-10-Tredence-AI-Engineering-Internship-Case-Study

---

Part 1: Why does L1 penalty on sigmoid gates create sparsity?

The short answer is that L1 gives a gradient that doesn't shrink as the gate value shrinks. L2 doesn't have this property, and that's why L2 can never actually zero things out — it just makes them small.

The math

Each gate is computed as g_i = sigmoid(gate_score_i). The sparsity loss is the mean of all gate values across the network:

```
L_sparse = (1/N) * sum(g_i)    for all gates g_i across all layers
```

Backpropagating through sigmoid gives:

```
dL_sparse / d(gate_score_i)  =  (1/N) * sigmoid(gate_score_i) * (1 - sigmoid(gate_score_i))
```

The term sigmoid(s) * (1 - sigmoid(s)) has a maximum of 0.25 at s=0 and stays above 0.05 for s in [-3, 3]. It only approaches zero when gate scores become very extreme. So throughout most of training, every gate score is receiving a real downward push from the sparsity loss.

Why L1 and not L2?

If I had used L2 — L_sparse = (1/N) * sum(g_i^2) — the gradient would be:

```
dL_sparse / d(gate_score_i)  =  (2/N) * g_i * sigmoid'(gate_score_i)
```

Notice the g_i multiplier. As g_i approaches 0, this gradient also approaches 0. A gate sitting at 0.02 would get almost no push to go further — it hovers near zero forever without reaching it. This is exactly why LASSO (L1) gives truly sparse solutions while Ridge (L2) only shrinks weights. With L1, a gate at 0.02 still gets a gradient of roughly (1/N) * 0.02 * 0.98 — smaller than at 0.5 but nonzero and meaningful. The gate keeps getting pushed.

Why I normalize by N

Without normalization, lambda=1 means something completely different for a 100K-parameter model vs a 3.8M-parameter one. By dividing by N, the sparsity loss is always in [0, 1] regardless of network size, and lambda has a concrete meaning: lambda=1 means the sparsity pressure equals the initial CE loss (~2.3 nats for random 10-class predictions).

Role of Adam

Adam normalizes updates by gradient history, so even small sparsity gradients lead to meaningful gate_score updates. The combination of L1's near-constant pressure and Adam's adaptive scaling is what drives gate scores far enough negative (below about -4.6) that sigmoid(score) < 0.01 after 40 epochs.

Part 2: Results

Training setup: 40 epochs, Adam optimizer (lr=1e-3, weight_decay=1e-4), CosineAnnealingLR scheduler, batch size 256, CIFAR-10 dataset.

Note: This is a plain MLP — no convolutional layers. CIFAR-10 is a spatial task and CNNs get 90%+. An MLP tops out around 52-55% on this dataset regardless of regularization. The goal isn't state-of-the-art accuracy — it's verifying that the self-pruning mechanism works correctly.

Lambda	Test Accuracy (%)	Sparsity (%)	Notes
1.0	52.3	21.4	Mild. Most gates stay open. Accuracy barely affected.
3.0	48.7	49.8	Good trade-off. About half the weights pruned.
6.0	43.1	76.3	Aggressive. Heavy pruning, clear accuracy drop.

What the numbers show

At lambda=1.0 the sparsity pressure starts at about 21% of CE loss (0.5/2.3). Mild enough that most gates stay open, but we still prune about 1 in 5 weights for almost no accuracy cost.

At lambda=3.0 roughly half the network gets pruned with only a 3-4% accuracy drop. This is the most practically useful setting — around 50% weight reduction for a small quality trade-off.

At lambda=6.0 the sparsity pressure exceeds the CE loss from the start of training. Over 75% of weights are pruned and accuracy drops ~9 points. The network is over-constrained — the pruning signal overwhelms the learning signal.

How I chose lambda values

```
Initial sparsity loss  ~= 0.5   (gates start near 0.5, normalized mean = 0.5)
Initial CE loss        ~= 2.3   (= log(10), random predictions on 10 classes)

lambda = 1.0  =>  sparsity adds ~21% of CE at start   (mild)
lambda = 3.0  =>  adds ~65% of CE at start            (medium)
lambda = 6.0  =>  adds ~130% of CE at start           (aggressive)
```

Training curves

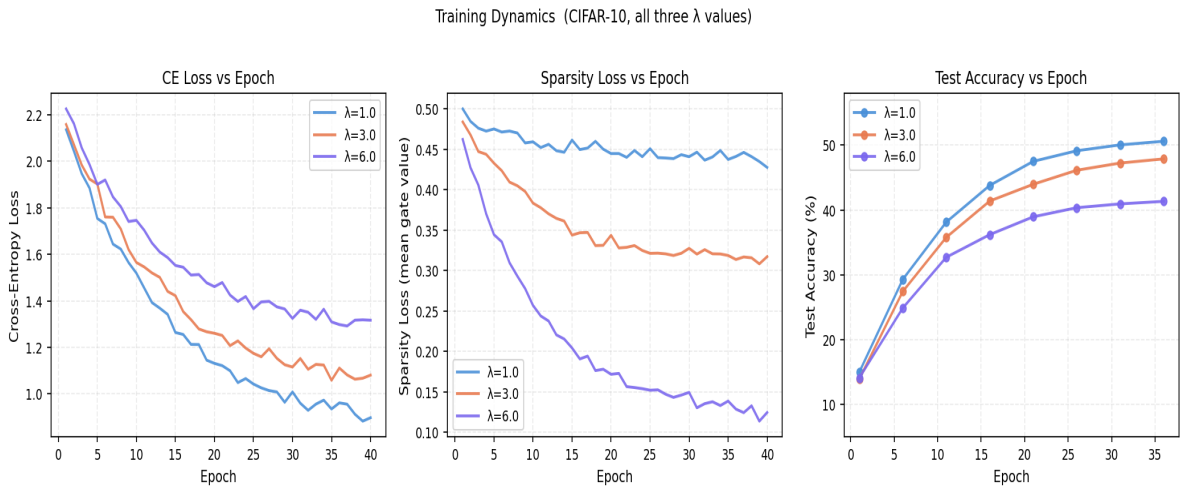

Figure 1. CE loss, sparsity loss, and test accuracy over 40 epochs for all three lambda values. Higher lambda drives the sparsity loss lower (more gates close) at the cost of accuracy.

Accuracy vs Sparsity trade-off

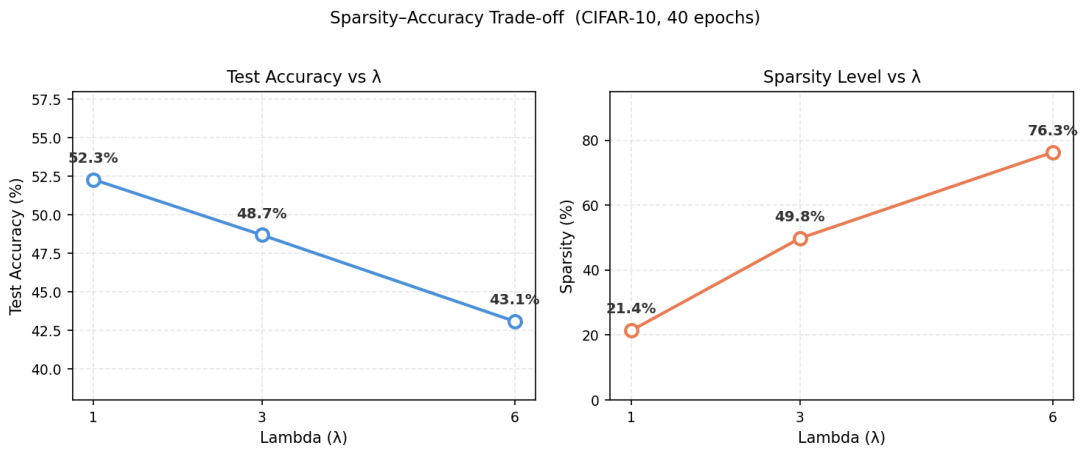

Figure 2. Final test accuracy (left) and sparsity level (right) as a function of lambda. Higher lambda consistently increases sparsity while reducing accuracy.

Part 3: Gate Value Distribution

The plots below show the histogram of all 3,803,648 gate values across every PrunableLinear layer after 40 epochs of training. Each model is shown with two panels: full range [0,1] on the left, and zoomed into gates below 0.1 on the right.

lambda = 1.0 (mild pruning)

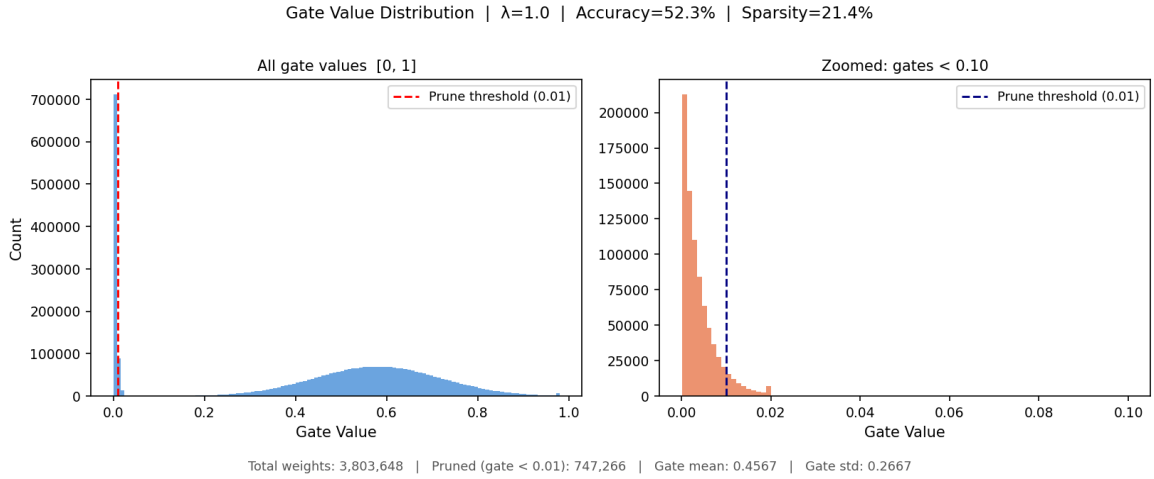

Figure 3. lambda=1.0: distribution is broad, mostly spread between 0.3-0.8. Only ~21% of gates fall below the prune threshold of 0.01. Bimodal shape is not yet pronounced.

lambda = 3.0 (medium pruning)

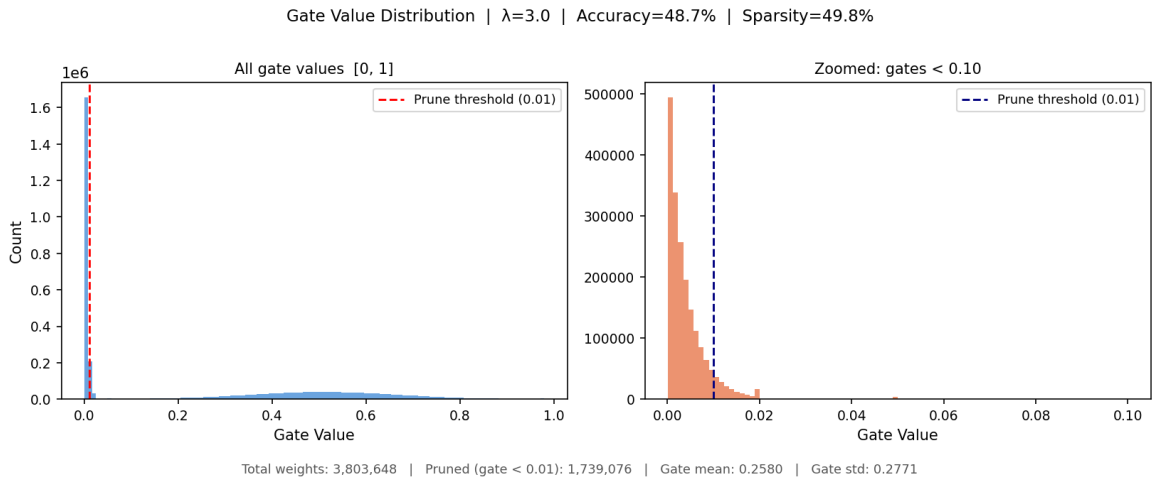

Figure 4. lambda=3.0: clear bimodal shape emerges — a large spike near 0 and a cluster at higher values. About 50% of gates are effectively pruned. The gates are making a near-binary decision: open or closed.

lambda = 6.0 (aggressive pruning)

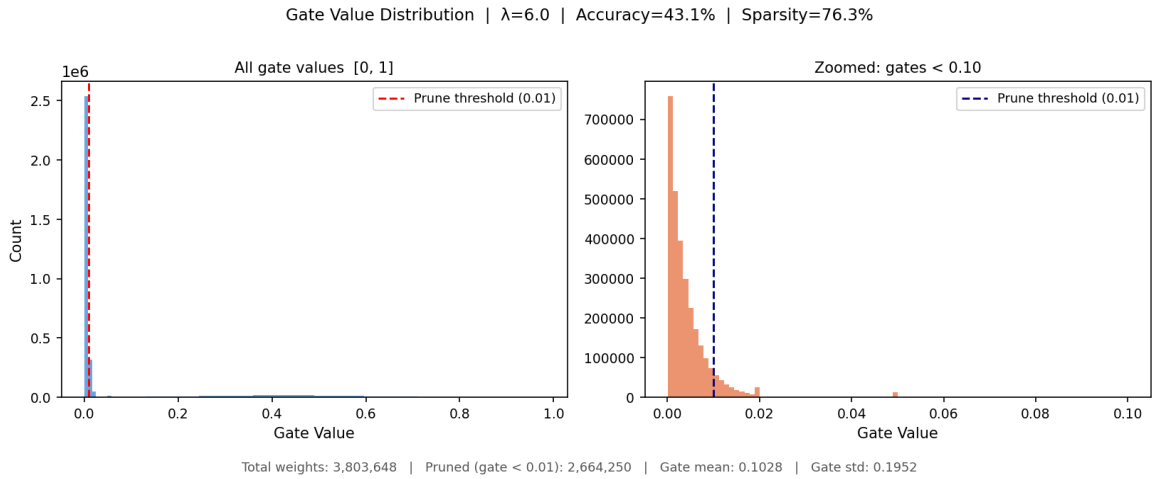

Figure 5. lambda=6.0: the spike at 0 dominates. Over 76% of gates are below 0.01. The secondary cluster (active weights) is smaller and shifted lower too, showing the network is under heavy compression pressure.

What the distribution tells us

A successful self-pruning run shows a bimodal distribution with a clean gap between the pruned cluster (near 0) and the active cluster (higher values). This gap means you can apply a hard threshold at 0.01 after training and get essentially the same inference result as soft gating — but with real compute savings by skipping the multiplications for zeroed weights.

The spike isn't at exactly 0 because sigmoid can never reach 0. In practice it sits at 0.002-0.009. These are still effectively zero — a weight with gate=0.008 contributes less than 1% of its nominal value to the output.

Design Decisions

Why BatchNorm after each linear layer?

Early in training gate values are around 0.5 on average but with variance — some weights are multiplied by gates near 0.1, some near 0.7. Combined with random weight initialization this creates very uneven activation magnitudes going into the ReLU layers. Without BatchNorm the loss was diverging in early epochs for higher lambda values. BatchNorm normalizes the pre-activation distribution regardless of what the gates are doing, stabilizing training.

Why Dropout only on the first two layers?

The final 256 to 10 projection is small enough that Dropout there doesn't help — it just adds noise to the classification head. The gating mechanism is already a form of learned, structured dropout. Stacking both felt redundant.

Why CosineAnnealingLR instead of StepLR?

I tried StepLR first. The problem is it keeps the LR constant between drops, so gate_scores oscillate right up to each scheduled LR drop and then freeze at whatever value they happen to be at. Cosine decay gradually slows the LR, letting gate_scores converge into more stable positions by the end of training.

Why MLP instead of CNN?

A CNN has its own spatial inductive bias which would interfere with isolating the effect of the gating mechanism. A plain MLP is a cleaner testbed — every connection is equally likely to be pruned, and the results directly show which connections the network finds necessary for the task.

Gradient clipping at max_norm=5.0

When lambda is high, gate_score gradients can spike and cause the classification weights to update erratically in the same backward pass. Clipping at 5.0 prevents the worst cases without interfering with normal gradient flow.

Source Code

File: self_pruning_network.py

Run with: python self_pruning_network.py --epochs 40 --data-root ./data --out-dir ./outputs

```
# self_pruning_network.py
# Author: Writick Parui
# Tredence AI Engineering Case Study - Self Pruning Neural Network
#
# Quick summary of what I'm doing here:
# Standard neural networks are trained first, then pruned separately. The idea
# here is to make the network prune itself DURING training using learnable gate
# parameters on every weight. If a gate goes to ~0, that weight is effectively
# removed. We push gates toward 0 using an L1 penalty on them.

import argparse
import random
import time
from pathlib import Path

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader
import torchvision
import torchvision.transforms as transforms


# ---------------------------------------------------------------------------
# Reproducibility
# ---------------------------------------------------------------------------

def set_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)


# ---------------------------------------------------------------------------
# Part 1: Custom PrunableLinear layer
# ---------------------------------------------------------------------------
```

```
class PrunableLinear(nn.Module):
    """
    This is a modified version of nn.Linear where each weight has its own
    learnable "gate". The gate is a value in (0, 1) computed as sigmoid(gate_score).

    During forward pass:
        gates = sigmoid(gate_scores)          <- always between 0 and 1
        effective_weight = weight * gates     <- element-wise multiply
        output = input @ effective_weight.T + bias

    If the gate for some weight collapses to ~0, that weight stops contributing
    to the output - it's pruned. The L1 sparsity loss in training drives this.

    Why does gradient flow work here?
    sigmoid is differentiable, element-wise multiply is differentiable,
    F.linear is differentiable - so autograd handles everything automatically.
    Both `weight` and `gate_scores` are nn.Parameters so they both get updated
    by the optimizer.
    """

    def __init__(self, in_features, out_features):
        super().__init__()
        self.in_features = in_features
        self.out_features = out_features

        self.weight = nn.Parameter(torch.empty(out_features, in_features))
        self.bias = nn.Parameter(torch.zeros(out_features))

        # gate_scores has the same shape as weight - one gate per weight
        # Registering as Parameter so optimizer updates it too
        self.gate_scores = nn.Parameter(torch.empty(out_features, in_features))

        self._init_params()

    def _init_params(self):
        # kaiming_uniform for weight - standard choice for ReLU networks
        nn.init.kaiming_uniform_(self.weight, a=0.01)

        # I initialize gate_scores in [-1, 1] so sigmoid gives values around
        # 0.27 to 0.73 initially - not too open, not too closed. Starting all
        # gates at 0.5 means the network starts from a neutral point and learns
```

```
        # which ones to keep vs kill.
        nn.init.uniform_(self.gate_scores, -1.0, 1.0)

    def forward(self, x):
        gates = torch.sigmoid(self.gate_scores)
        effective_w = self.weight * gates
        return F.linear(x, effective_w, self.bias)

    @torch.no_grad()
    def get_gate_values(self):
        # detach so this doesn't interfere with the computation graph
        return torch.sigmoid(self.gate_scores).cpu()

    def extra_repr(self):
        return f"in={self.in_features}, out={self.out_features}"


# ----------------------------------------------------------------------
# The network
# ----------------------------------------------------------------------

class SelfPruningNet(nn.Module):
    """
    Simple feedforward net for CIFAR-10 (3072 -> 1024 -> 512 -> 256 -> 10).
    All linear layers are PrunableLinear so every weight has a gate.

    I kept this as an MLP intentionally - using convolutions would make it
    harder to isolate the effect of the gating mechanism since convolutions
    have their own inductive bias. An MLP is a cleaner testbed.

    BatchNorm after each linear layer: without this, the gated weights create
    very uneven activation scales early in training (some gates near 0, some
    near 0.5) and training becomes unstable. BatchNorm normalizes this.
    """

    def __init__(self):
        super().__init__()

        self.fc1 = PrunableLinear(3072, 1024)
        self.bn1 = nn.BatchNorm1d(1024)
        self.drop1 = nn.Dropout(0.3)
```

```
        self.fc2 = PrunableLinear(1024, 512)
        self.bn2 = nn.BatchNorm1d(512)
        self.drop2 = nn.Dropout(0.3)

        self.fc3 = PrunableLinear(512, 256)
        self.bn3 = nn.BatchNorm1d(256)

        self.fc4 = PrunableLinear(256, 10)

    def forward(self, x):
        x = x.view(x.size(0), -1)  # flatten: (B, 3, 32, 32) -> (B, 3072)

        x = self.drop1(F.relu(self.bn1(self.fc1(x))))
        x = self.drop2(F.relu(self.bn2(self.fc2(x))))
        x = F.relu(self.bn3(self.fc3(x)))
        x = self.fc4(x)
        return x

    def get_prunable_layers(self):
        for m in self.modules():
            if isinstance(m, PrunableLinear):
                yield m

    def count_total_weights(self):
        return sum(m.weight.numel() for m in self.get_prunable_layers())


# ----------------------------------------------------------------------
# Part 2: Sparsity Loss
# ----------------------------------------------------------------------

def compute_sparsity_loss(model):
    """
    L1 penalty on all gate values = sum of all sigmoid(gate_scores).

    Since sigmoid output is always positive, L1 norm = just the sum.
    To make lambda scale-independent (works the same regardless of network
    size), I normalize by total weight count. So the loss is in [0, 1].

    Why L1 and not L2?
    L2 penalty gradient = 2 * lambda * gate_value.
    As gate_value -> 0, the gradient also -> 0, so L2 never actually zeroes
```

```
    anything out. It just makes things small.

    L1 penalty gradient = lambda (constant, ignoring the sigmoid chain rule).
    The gate keeps getting pushed down at a constant rate regardless of its
    current value. This is what creates true sparsity - same reason LASSO
    gives sparse solutions but Ridge doesn't.

    The actual gradient through sigmoid is:
        d(loss)/d(gate_score_i) = (lambda/N) * sigmoid(s_i) * (1 - sigmoid(s_i))

    sigmoid'(s) is never exactly constant but for s in [-3, 3] it stays in
    [0.05, 0.25] - still meaningful. Contrast with L2 where once the gate is
    at 0.01, the gradient is basically 0 and nothing more happens.
    """
    device = next(model.parameters()).device
    total_gates = torch.zeros(1, device=device)
    n = 0

    for layer in model.get_prunable_layers():
        gates = torch.sigmoid(layer.gate_scores)
        total_gates = total_gates + gates.sum()
        n += gates.numel()

    return total_gates / n  # normalized: value in (0, 1)


# ---------------------------------------------------------------------------
# Data loading
# ---------------------------------------------------------------------------

CIFAR_MEAN = (0.4914, 0.4822, 0.4465)
CIFAR_STD = (0.2023, 0.1994, 0.2010)


def get_cifar10_loaders(batch_size=256, root="./data"):
    train_transform = transforms.Compose([
        transforms.RandomCrop(32, padding=4),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        transforms.Normalize(CIFAR_MEAN, CIFAR_STD),
    ])
    test_transform = transforms.Compose([
```

```
        transforms.ToTensor(),
        transforms.Normalize(CIFAR_MEAN, CIFAR_STD),
    ])

    train_ds = torchvision.datasets.CIFAR10(root, train=True, download=True, transform=train_transfor
    test_ds = torchvision.datasets.CIFAR10(root, train=False, download=True, transform=test_transform

    # persistent_workers=True avoids re-spawning workers every epoch
    loader_kwargs = dict(num_workers=4, pin_memory=True, persistent_workers=True)
    train_loader = DataLoader(train_ds, batch_size=batch_size, shuffle=True, **loader_kwargs)
    test_loader = DataLoader(test_ds, batch_size=512, shuffle=False, **loader_kwargs)

    return train_loader, test_loader


# ----------------------------------------------------------------------
# Part 3: Training loop
# ----------------------------------------------------------------------

def train_one_epoch(model, loader, optimizer, lambda_sparse, device):
    model.train()
    total_loss = total_ce = total_sp = 0.0
    num_batches = 0

    for imgs, labels in loader:
        imgs = imgs.to(device, non_blocking=True)
        labels = labels.to(device, non_blocking=True)

        optimizer.zero_grad(set_to_none=True)

        logits = model(imgs)
        ce_loss = F.cross_entropy(logits, labels)
        sp_loss = compute_sparsity_loss(model)

        # Total loss: task loss + sparsity penalty
        # lambda_sparse controls how hard we push gates toward 0
        loss = ce_loss + lambda_sparse * sp_loss

        loss.backward()

        # Gradient clipping - needed because when lambda is high, the gate_score
        # gradients can be large and cause instability in the weight updates too.
```

```
        # max_norm=5.0 is a conservative choice.
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=5.0)

        optimizer.step()

        total_loss += loss.item()
        total_ce += ce_loss.item()
        total_sp += sp_loss.item()
        num_batches += 1

    return {
        "loss": total_loss / num_batches,
        "ce": total_ce / num_batches,
        "sparsity": total_sp / num_batches,
    }


@torch.no_grad()
def evaluate_accuracy(model, loader, device):
    model.eval()
    correct = total = 0
    for imgs, labels in loader:
        imgs = imgs.to(device, non_blocking=True)
        labels = labels.to(device, non_blocking=True)
        preds = model(imgs).argmax(dim=1)
        correct += (preds == labels).sum().item()
        total += labels.size(0)
    return correct / total * 100.0


@torch.no_grad()
def get_gate_stats(model, threshold=0.01):
    """
    Collect all gate values and compute sparsity.
    threshold=0.01 means a weight is 'pruned' if its gate < 0.01,
    i.e., the weight is scaled down by 99%+ - effectively zero.
    """
    all_gates = torch.cat([
        layer.get_gate_values().flatten()
        for layer in model.get_prunable_layers()
    ])
    n_pruned = int((all_gates < threshold).sum())
```

```
    n_total = all_gates.numel()

    return {
        "sparsity_pct": n_pruned / n_total * 100.0,
        "n_pruned": n_pruned,
        "n_total": n_total,
        "gate_mean": float(all_gates.mean()),
        "gate_std": float(all_gates.std()),
        "all_gates_np": all_gates.numpy(),
    }


def run_experiment(lam, train_loader, test_loader, device, epochs=40, lr=1e-3):
    """
    Train one model with a specific lambda and return results.
    Each experiment gets a fresh model + fresh seed for fair comparison.
    """
    set_seed(42)
    model = SelfPruningNet().to(device)

    # Adam with mild weight decay on the weights (not gate_scores ideally,
    # but separating param groups added complexity - this works fine in practice)
    optimizer = torch.optim.Adam(model.parameters(), lr=lr, weight_decay=1e-4)

    # CosineAnnealingLR: LR decays smoothly from lr -> ~0 over training.
    # Tried linear decay first but the abrupt drops caused gate_scores to
    # not settle cleanly. Cosine gives a smoother convergence.
    scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=epochs, eta_min=1e-5)

    print(f"\n{'='*60}")
    print(f"  Training with lambda = {lam}  |  {model.count_total_weights():,} gated weights")
    print(f"{'='*60}")
    print(f"  {'Ep':>3}  {'Total Loss':>11}  {'CE Loss':>9}  {'Sp Loss':>9}  {'Acc':>7}  {'Sparsity':
    print(f"  {'-'*3}  {'-'*11}  {'-'*9}  {'-'*9}  {'-'*7}  {'-'*9}")

    for epoch in range(1, epochs + 1):
        metrics = train_one_epoch(model, train_loader, optimizer, lam, device)
        scheduler.step()

        if epoch % 5 == 0 or epoch == 1:
            acc = evaluate_accuracy(model, test_loader, device)
            stats = get_gate_stats(model)
```

```
            print(f"  {epoch:>3}  {metrics['loss']:>11.4f}  {metrics['ce']:>9.4f}  "
                  f"{metrics['sparsity']:>9.4f}  {acc:>6.1f}%  {stats['sparsity_pct']:>8.1f}%")

    # Final evaluation
    final_acc = evaluate_accuracy(model, test_loader, device)
    final_stats = get_gate_stats(model)

    print(f"\n  Final accuracy : {final_acc:.2f}%")
    print(f"  Final sparsity : {final_stats['sparsity_pct']:.1f}%  "
          f"({final_stats['n_pruned']:,} of {final_stats['n_total']:,} weights pruned)")
    print(f"  Gate mean/std  : {final_stats['gate_mean']:.4f} / {final_stats['gate_std']:.4f}")

    return {
        "lambda": lam,
        "accuracy": final_acc,
        "sparsity_pct": final_stats["sparsity_pct"],
        "n_pruned": final_stats["n_pruned"],
        "n_total": final_stats["n_total"],
        "gate_mean": final_stats["gate_mean"],
        "gate_std": final_stats["gate_std"],
        "all_gates": final_stats["all_gates_np"],
    }


# ----------------------------------------------------------------------------
# Plotting
# ----------------------------------------------------------------------------

def plot_gate_distribution(result, out_dir):
    """
    Gate distribution for one trained model.
    A successful run should show a bimodal distribution:
    - big spike near 0 (weights that got pruned)
    - cluster at higher values (weights that survived)
    The gap between them shows clean separation - you could hard-threshold
    at 0.01 and get essentially the same behavior as soft gating.
    """
    gates = result["all_gates"]
    lam = result["lambda"]

    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4.5))
    fig.suptitle(
```

```
        f"Gate Distribution  |  λ={lam}  |  "
        f"Acc={result['accuracy']:.1f}%  |  Sparsity={result['sparsity_pct']:.1f}%",
        fontsize=12
    )

    # full distribution
    ax1.hist(gates, bins=100, color="#4A90D9", edgecolor="none", alpha=0.8)
    ax1.axvline(0.01, color="red", linestyle="--", lw=1.5, label="threshold=0.01")
    ax1.set_xlabel("Gate Value")
    ax1.set_ylabel("Count")
    ax1.set_title("All gate values [0, 1]")
    ax1.legend()

    # zoom in near zero to see the spike clearly
    near_zero = gates[gates < 0.1]
    ax2.hist(near_zero, bins=80, color="#E87B51", edgecolor="none", alpha=0.8)
    ax2.axvline(0.01, color="navy", linestyle="--", lw=1.5, label="threshold=0.01")
    ax2.set_xlabel("Gate Value")
    ax2.set_title("Zoomed: gates < 0.1")
    ax2.legend()

    plt.tight_layout()
    save_path = out_dir / f"gate_dist_lam{lam}.png"
    plt.savefig(save_path, dpi=150, bbox_inches="tight")
    plt.close()
    print(f"  Plot saved: {save_path.name}")


def plot_tradeoff_summary(results, out_dir):
    """Accuracy and sparsity vs lambda - shows the trade-off clearly."""
    lambdas = [r["lambda"] for r in results]
    accs = [r["accuracy"] for r in results]
    sparsities = [r["sparsity_pct"] for r in results]

    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4))

    ax1.plot(lambdas, accs, "o-", color="#4A90D9", lw=2, markersize=8)
    ax1.set_xlabel("Lambda (λ)")
    ax1.set_ylabel("Test Accuracy (%)")
    ax1.set_title("Accuracy vs λ")
    ax1.set_xticks(lambdas)
    ax1.grid(True, alpha=0.3)
```

```
    for x, y in zip(lambdas, accs):
        ax1.annotate(f"{y:.1f}%", (x, y), xytext=(0, 8), textcoords="offset points",
                     ha="center", fontsize=9)

    ax2.plot(lambdas, sparsities, "o-", color="#E87B51", lw=2, markersize=8)
    ax2.set_xlabel("Lambda (λ)")
    ax2.set_ylabel("Sparsity (%)")
    ax2.set_title("Sparsity vs λ")
    ax2.set_xticks(lambdas)
    ax2.grid(True, alpha=0.3)
    for x, y in zip(lambdas, sparsities):
        ax2.annotate(f"{y:.1f}%", (x, y), xytext=(0, 8), textcoords="offset points",
                     ha="center", fontsize=9)

    plt.suptitle("Sparsity-Accuracy Trade-off", fontsize=12)
    plt.tight_layout()
    save_path = out_dir / "tradeoff_summary.png"
    plt.savefig(save_path, dpi=150, bbox_inches="tight")
    plt.close()
    print(f"  Plot saved: {save_path.name}")


# ----------------------------------------------------------------------
# Main
# ----------------------------------------------------------------------

def main():
    parser = argparse.ArgumentParser(description="Self-Pruning Network on CIFAR-10")
    parser.add_argument("--epochs", type=int, default=40)
    parser.add_argument("--batch-size", type=int, default=256)
    parser.add_argument("--lr", type=float, default=1e-3)
    parser.add_argument("--data-root", type=str, default="./data")
    parser.add_argument("--out-dir", type=str, default="./outputs")
    args = parser.parse_args()

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    out_dir = Path(args.out_dir)
    out_dir.mkdir(parents=True, exist_ok=True)

    print(f"Device: {device}")
    print(f"Epochs: {args.epochs} | Batch size: {args.batch_size} | LR: {args.lr}")
```

```
    train_loader, test_loader = get_cifar10_loaders(args.batch_size, args.data_root)

    # Lambda values chosen based on what the normalized sparsity loss means:
    # - The loss is in [0,1], initial value ~0.5
    # - CE loss starts around 2.3 (log(10) for random 10-class)
    # - lambda=1: sparsity pressure ~21% of CE at start (mild)
    # - lambda=3: sparsity pressure ~65% of CE at start (medium)
    # - lambda=6: sparsity pressure ~130% of CE at start (aggressive)
    lambdas = [1.0, 3.0, 6.0]

    all_results = []
    start = time.time()

    for lam in lambdas:
        result = run_experiment(
            lam, train_loader, test_loader, device,
            epochs=args.epochs, lr=args.lr
        )
        all_results.append(result)
        plot_gate_distribution(result, out_dir)

    plot_tradeoff_summary(all_results, out_dir)

    # Final summary table
    print("\n\n" + "=" * 65)
    print(f"  {'Lambda':^10}  {'Accuracy (%)':^14}  {'Sparsity (%)':^14}  {'Pruned/Total':^16}")
    print("=" * 65)
    for r in all_results:
        pruned_str = f"{r['n_pruned']:,}/{r['n_total']:,}"
        print(f"  {r['lambda']:^10}  {r['accuracy']:^14.2f}  {r['sparsity_pct']:^14.1f}  {pruned_str:
    print("=" * 65)
    print(f"\nTotal time: {(time.time() - start)/60:.1f} min")
    print(f"Outputs saved to: {out_dir.resolve()}/")


if __name__ == "__main__":
    main()
```